Data Science Lab

Name: Vishal Rajesh Mahajan | Experiment No: 6A

Class: BE INFT B-1 | Roll No: 1

Importing Dataset from Kaggle

```
import kagglehub
ananthr1_weather_prediction_path =
kagglehub.dataset_download('ananthr1/weather-prediction')
print('Data source import complete.')
```

Loading and Inspecting Dataset

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
df = pd.read_csv('/kaggle/input/weather-prediction/seattle-weather.csv')
df.head()
```

	date	precipitation	temp_max	temp_min	wind	weather
0	2012-01-01	0.0	12.8	5.0	4.7	drizzle
1	2012-01-02	10.9	10.6	2.8	4.5	rain
2	2012-01-03	0.8	11.7	7.2	2.3	rain
3	2012-01-04	20.3	12.2	5.6	4.7	rain
4	2012-01-05	1.3	8.9	2.8	6.1	rain

Checking for Missing or Duplicate Values

```
df.isnull().sum()
```

	0
date	0
precipitation	0
temp_max	0
temp_min	0
wind	0
weather	0

Extracting Temperature Data for Training

```
training_set = df.iloc[:,2:3].values
training_set
```

```
array([[12.8],
       [10.6],
       [11.7],
       ...,
       [ 7.2],
       [ 5.6],
       [ 5.6]])
```

Creating Training Sequences (Sliding Window Method)

```
def df_to_XY(df,window_size=10):
 X_train=[]
 y_train=[]

 for i in range(10,len(training_set)):
   X_train.append(training_set[i-10:i,0])
   y_train.append(training_set[i,0])

 X_train, y_train = np.array(X_train), np.array(y_train)
 return X_train, y_train
```

Splitting Data into Train, Validation and Test Sets

```
WINDOW = 10
X,y = df_to_XY(df,WINDOW)
print(len(X),len(y))
X_train = X[:800]
y_train = y[:800]
X_val = X[800:1000]
y_val = y[800:1000]
X_test = X[1000:]
x_test = y[1000:]
```

Reshaping Data for RNN Input

```
X_train = np.reshape(X_train,(X_train.shape[0],X_train.shape[1],1))
X_val = np.reshape(X_val,(X_val.shape[0],X_val.shape[1],1))
X_test = np.reshape(X_test,(X_test.shape[0],X_test.shape[1],1))
```

Building the RNN Model

```
from keras.models import Sequential
from keras.layers import Dense, LSTM, Dropout

regressor = Sequential()

regressor.add(LSTM(units=50, return_sequences = True, input_shape=(X_train.shape[1], 1)))
regressor.add(Dropout(0.2))
regressor.add(LSTM(units=50, return_sequences = True))
regressor.add(Dropout(0.2))
regressor.add(LSTM(units=50, return_sequences = True))
regressor.add(Dropout(0.2))
regressor.add(LSTM(units=50))
regressor.add(Dropout(0.2))
regressor.add(Dense(units=1))
```

Compiling and Training the Model

```
regressor.compile(optimizer='adam',loss='mean_squared_error')

from tensorflow.keras.callbacks import ModelCheckpoint, EarlyStopping
history=regressor.fit(X_train,y_train,validation_data=(X_val,y_val),epochs=100, batch_size=32)
```

```
Epoch 1/100
25/25 ━━━━━━━━━━━━━━━━━━━━ 8s 99ms/step - loss: 237.0830 - val_loss: 223.6034
Epoch 2/100
25/25 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 87.2720 - val_loss: 139.5410
Epoch 3/100
25/25 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 67.2129 - val_loss: 114.8826

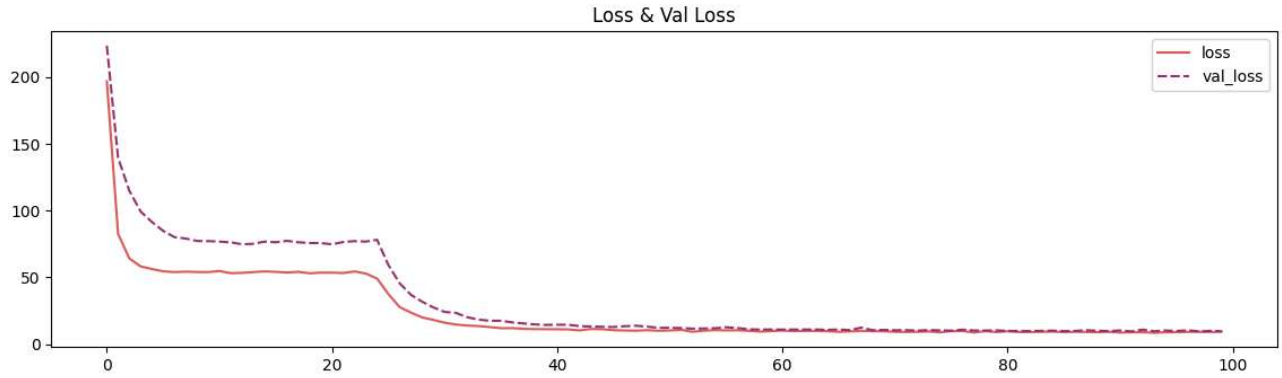
25/25 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 9.1830 - val_loss: 10.2558
Epoch 98/100
25/25 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 9.3576 - val_loss: 9.4837
Epoch 99/100
25/25 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 9.0405 - val_loss: 9.8662
Epoch 100/100
25/25 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 8.8721 - val_loss: 9.6044
```

Visualizing Training and Validating Loss

```
his = pd.DataFrame(history.history)

import seaborn as sns
his.columns
history_loss = his[['loss', 'val_loss']]

fig,axes = plt.subplots(2,1,figsize=(14,8))
plt.subplot(2,1,1)
plt.title("Loss & Val Loss")
sns.lineplot(history_loss,palette="flare");
```



Making Prediction on Train, Validation and Test Data

```
train_pred = regressor.predict(X_train).flatten()
val_pred = regressor.predict(X_val).flatten()
test_pred = regressor.predict(X_test).flatten()
```

```
25/25 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 51ms/step
15/15 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step
```

Combining Predictions with Actual Data

```
pred = np.concatenate([train_pred,val_pred,test_pred])
df_pred = pd.DataFrame(df["temp_max"].copy())
df_pred.columns=["actual"]
df_pred = df_pred[WINDOW:]
df_pred["predicted"] = pred
```

Plotting Validation and Test Results

```
fig,axes = plt.subplots(2,1,figsize=(14,8),dpi=400)

plt.subplot(2,1,1)
plt.title("Validation Results")
sns.lineplot(df_pred[800:],alpha=0.8,palette="flare",linestyle=None);

plt.subplot(2,1,2)
plt.title("Test Results")
sns.lineplot(df_pred[1000:],alpha=0.8,palette="flare",linestyle=None);
```

